

Software Requirements Specification

Fidel Tutorial — Public Website Upgrade

Prepared for: Fidel Tutorial (fideltutorial.com)

Document version 1.0 — July 2026

Scope: Marketing/public website

1. Introduction

1.1 Purpose

This document specifies the requirements for upgrading the Fidel Tutorial public marketing website (currently a WordPress site at fideltutorial.com) to a new platform. A static HTML/CSS redesign has already been produced covering visual design and page copy; this SRS defines the functional and non-functional requirements needed to turn that design into a working, maintainable, revenue-capable website, and recommends the supporting architecture.

1.2 Scope

The system covers the public-facing marketing site only: home page, program pages, institutional/partnership pages, diaspora page, blog/resources, tutor directory, testimonials, FAQ, lead-capture and trial booking, and payment collection for packages/bootcamps. It explicitly excludes the Fidel Academy LMS (academy.fideltutorial.com) and the Tutor Management & Automation System, though the LMS is a linked integration point.

1.3 Current State Assessment

The live site (fideltutorial.com) is built on WordPress with a page-builder theme. Findings from review:

- Content is static per page; tutor bios, testimonials, and statistics are hand-edited in the page builder.
- A results/statistics section renders a visible rendering bug (concatenated placeholder digits instead of animated counters).
- No blog posts appear currently indexed despite a Blog nav item.
- Lead capture is a single generic contact form with no visible destination workflow, and a separate 'Register/Book Tutor' page.
- No payment collection, no diaspora-specific USD flow, no LMS account linkage from the marketing site.

A new static HTML/CSS redesign (Fraunces/Manrope typography, moss-green and gold palette) has been produced and is visually and structurally strong: it adds audience-segmented sections (parents, schools, diaspora, NGOs/donors), a flagship bootcamp callout, an FAQ, and a richer footer/nav structure. However, it is entirely front-end: no forms submit anywhere (CTAs are mailto:/tel: links), all tutor and testimonial content is placeholder, and there is no CMS, payment, or backend of any kind.

2. Overall Description

2.1 Product Perspective

The upgraded website will be a standalone React + Express + MySQL application, deployed independently from the Fidel Academy LMS but sharing the same visual/brand system as the new design and, where the host allows it, the same git-based

deployment habits already used across our projects. It will integrate with the LMS via a login link (and later, optionally, shared authentication).

2.2 User Classes

User class	Needs
Prospective parent / student	Understand programs, see pricing/trust signals, book a free trial, pay for a package
Diaspora sponsor	Pay in USD/card for a relative's tutoring, see diaspora-specific programs
School / NGO partner	Request a partnership brief, understand institutional offerings
Prospective tutor	Apply to join the tutor network
Marketing/admin staff	Edit tutors, testimonials, blog posts, FAQ, and view/export leads — without a developer
Site administrator (dev)	Manage users, payment reconciliation, technical configuration

2.3 Assumptions & Constraints

- Payment collection must support Telebirr, CBE Birr, and international cards (for diaspora/USD).
- Placeholder tutor bios, photos, and testimonials in the new design must be replaced with real, consented content before go-live.

3. Functional Requirements

3.1 Public Pages (from the new design, retained)

- Home, Programs, Institutions/For Schools, Diaspora, Tutors, Testimonials, FAQ/Resources, Contact - content and visual design as already drafted.
- Each program and institutional offering should resolve to its own indexable URL (e.g. /programs/exam-bootcamps) rather than an in-page anchor only, for SEO.

3.2 Lead Capture & Trial Booking (new)

- FR-1: The 'Book a Free Trial' CTA opens a form capturing: parent/student name, phone, email, grade, subject(s) of interest, delivery preference (online/in-home), and city/country.
- FR-2: On submission, the lead is stored in the database and a notification email is sent to the admissions inbox.
- FR-3: Admin staff can view, filter, and export captured leads from the admin panel.
- FR-4: Forms are protected against spam/bot submission (honeypot field and/or rate limiting; CAPTCHA optional).

3.3 Content Management (new)

- FR-5: Marketing/admin staff can create, edit, publish, and unpublish: tutor profiles (photo, bio, subjects, experience), testimonials, blog/resource posts, and FAQ entries - without code changes.
- FR-6: Blog posts support a title, cover image, rich-text body, publish date, and category, and are served at indexable URLs (e.g. /blog/slug).
- FR-7: Downloadable resources (past papers, mock exams) can be uploaded and optionally gated behind an email/phone capture before download.

3.4 Payments (new)

- FR-8: Parents/sponsors can pay for a tutoring package or bootcamp seat online via Telebirr, CBE Birr, or card, in Birr.

- FR-9: Diaspora sponsors can pay the same offerings in USD via international card.
- FR-10: On successful payment, the system records the transaction, sends a confirmation email/receipt, and notifies admissions by email to trigger tutor matching.
- FR-11: Failed/pending payments are visible to admin staff for follow-up.

3.5 LMS & Tutor-System Integration

- FR-12: A 'Student LMS Login' link directs to academy.fideltutorial.com. (Stretch goal, out of initial scope: shared session/SSO between the marketing site and the LMS.)
- FR-13: New leads and paid bookings are visible to the Tutor Management System's staff for assignment (via shared database access or an internal API call — exact mechanism to be confirmed with that system's design).

3.6 Administration

- FR-14: Role-based admin access (e.g. Super Admin, Content Editor, Admissions) via a lightweight admin panel.
- FR-15: Basic analytics dashboard or at minimum Google Analytics / Search Console integration.

4. Non-Functional Requirements

Category	Requirement
Performance	Home page first contentful paint under ~2s on a typical Ethiopian mobile connection; images optimized/lazy-loaded.
SEO	Server-rendered HTML for all public pages; per-page meta tags, sitemap.xml, robots.txt, schema.org EducationalOrganization/LocalBusiness markup.
Accessibility	Maintain the ARIA roles and semantic structure already present in the new design (WCAG 2.1 AA as a target).
Localization	Full English/Amharic content toggle (not decorative phrases only), extensible to future languages.
Security	HTTPS everywhere, CSRF protection on forms, hashed credentials, PCI-compliant payment handling via the gateway (no raw card data touches the app server).
Reliability	Automated daily database backups; deployment rollback procedure consistent with the existing LMS git workflow.
Maintainability	Content editable by non-technical staff through the admin panel without a code deployment.
Compatibility	Fully responsive across mobile, tablet, and desktop; supported on current versions of Chrome, Safari, and Firefox.

5. Recommended Architecture & Technology Stack

5.1 Rationale

The recommendation is a React-based stack (React frontend, Express API, MySQL database), matching the pattern (React + Express + MySQL) so the team works in one language and one set of conventions across projects. The one deviation from a plain React SPA is using Next.js rather than Vite: organic search visibility is a core requirement for a marketing site, and a client-rendered SPA sends crawlers a near-empty HTML shell until JavaScript runs. Next.js keeps the same React components and hooks the team already knows, but renders real HTML on the server/at build time, matching the indexability the current WordPress site already has.

5.2 Recommended Stack

Layer	Recommendation	Why
Frontend	Next.js + TypeScript + Tailwind CSS	React model, with server rendering for SEO
Backend API	Node.js + Express + TypeScript	As planned; kept as a separate service the frontend calls
Database	MySQL, accessed via Prisma ORM	Matches existing MySQL/MariaDB environment; Prisma adds type-safe queries and migrations for the more complex, evolving data model here (tutors, posts, payments)
Validation	Zod, shared between frontend and backend	One schema definition prevents form/API mismatches
File uploads	Multer + Sharp	Multer, Sharp auto-generates optimized thumbnails for tutor photos and blog covers
Admin panel	React Admin (or a lean custom dashboard) against the Express API	Fast CRUD screens for tutors, testimonials, blog, and leads without hand-building forms
Auth (admin only)	JWT (access + refresh token) or session store in MySQL	Public pages stay open; only the admin panel requires login
Payments	Chapa or SantimPay (official Node SDKs)	Single integration covering Telebirr, CBE Birr, and international cards for USD/diaspora payments
Email	Nodemailer via SMTP or a transactional provider	Trial-booking confirmations, payment receipts, admin lead notifications
Process management / deployment	PM2 on the host; git-based deploy where supported	Node equivalent of the git-pull workflow already used elsewhere; confirm the host (e.g. cPanel 'Setup Node.js App'/Passenger) supports a persistent Node process before committing to this stack
SEO/analytics	next-sitemap, schema.org JSON-LD (EducationalOrganization), GA4 or Plausible	Low-effort additions that meaningfully help discoverability

5.3 Repository Structure

A single repository with /client (Next.js) and /server (Express) directories, plus a shared /shared directory for Zod schemas and TypeScript types, mirrors the project layout and keeps the two sides from drifting out of sync on data shapes.

5.4 Hosting Consideration

This stack requires a persistent Node.js process. Before committing, confirm the production host (Yegara Host or otherwise) offers a Node.js application feature (commonly available via cPanel's 'Setup Node.js App'/Passenger) or budget for a small VPS if it does not.

6. High-Level Data Model

Entity	Key fields
leads	name, phone, email, grade, subject, delivery_mode, city, source, status, created_at
tutors	name, photo, subjects, bio, experience_years, is_published
testimonials	author_name, role/location, quote, is_published

blog_posts	title, slug, cover_image, body, category, published_at
resources	title, file, is_gated
programs	name, slug, description, category (tutoring/bootcamp/camp/test-prep/counselling)
payments	lead_id, program_id, amount, currency, gateway, gateway_ref, status
admin_users	name, email, role, password_hash, refresh_token